LUDOVIC POTVIN
Baccalauréat en informatique

Rapport de TP3

Rapport
présenté à
Julien Maitre

Dans le cadre du cours
8INF700 – Sujet special en informatique

Département d'informatique
Université du Québec à Chicoutimi
10 mars 2026

Introduction

Le but de ce travail est de comparer l'approche par transfert d'apprentissage avec la construction d'une architecture CNN personnalisée aux besoins. Pour ce travail, j'ai choisi le jeu de données «AI Generated Images vs Real Images»[1]. L'objectif, à l'aide de ce jeu de données, était d'entraîner les modèles à faire la différence entre des images réelles et des images générées par intelligence artificielle. J'ai choisi le modèle ResNet-50 pour effectuer le transfert d'apprentissage. Ensuite, j'ai construit ma propre architecture CNN pour tenter de répondre aux besoins. Finalement, j'ai comparé les deux approches afin d'observer les différences entre elles.

Méthodologie générale

Le projet a été structuré de manière à comparer les modèles sur les mêmes bases. Les deux modèles partagent plusieurs éléments communs, comme la *loss function*, l'optimiseur et la séparation des données. Le projet est découpé en deux modules. Un contient les modèles qui suivent la même structure et l'autre contient le code pour l'apprentissage. À chaque *epoch*, le programme enregistre le modèle. Il est aussi possible de tester les modèles en indiquant le nom du fichier à charger dans le fichier config.py.

Pour entraîner les modèles, j'ai choisi la fonction de perte *CrossEntropyLoss*. J'ai fait ce choix car j'étais familier avec elle. Un des défauts de *CrossEntropyLoss* est qu'elle a tendance à biaiser le modèle lorsque le jeu de données est déséquilibré. Cependant, mon jeu de données est plutôt équilibré, ce qui atténue cet inconvénient.

Ensuite, j'ai choisi SGD comme optimiseur. J'ai initialement fait ce choix car c'est l'optimiseur utilisé pour entraîner ResNet. Je ne pense pas que ce choix soit idéal pour l'architecture CNN personnalisée. Lorsque j'ai tenté d'entraîner mon CNN, il s'est avéré difficile de faire converger l'optimiseur vers une solution stable, celui-ci se déséquilibrant constamment à chaque *epoch*. Je tenais cependant à garder le même optimiseur pour les deux modèles, en raison de la nature comparative du travail.

Finalement, les ensembles de données sont chargés en utilisant la même seed lors de l'entraînement des deux modèles. Ils ont donc eu accès exactement au même jeu d'entraînement, de validation et de test.

Méthodologie *transfer learning*

Pour le *transfer learning*, j'ai choisi le modèle ResNet-50. Un grand avantage de ce modèle est la présence d'une seule couche FC. Je n'ai donc eu qu'à modifier la couche finale, qui est de 2048 vers 1000 dans le modèle original. La couche FC que j'ai choisie est une couche linéaire de 2048 vers 2, afin de permettre au modèle de classifier une image comme étant soit générée par IA, soit réelle.

Initialement, je n'effectuais que 5 *epoch* lors de l'entraînement. J'ai cependant constaté que ce nombre n'était pas suffisant pour atteindre des résultats convaincants. J'ai ensuite remarqué, grâce aux graphiques, que le *learning rate* était trop élevé. En effet, la courbe d'entraînement montait très rapidement, mais

1 https://www.kaggle.com/datasets/cashbowman/ai-generated-images-vs-real-images

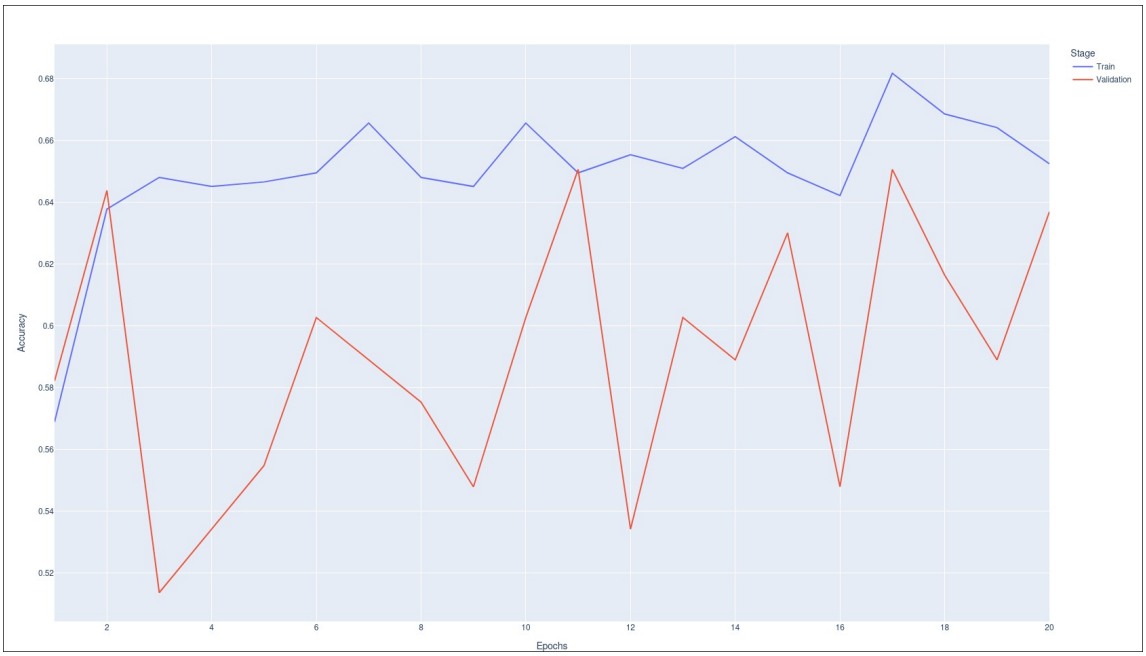
stagnait lors des *epoch* suivantes. J'ai obtenu les meilleurs résultats avec un taux d'apprentissage de 0,001 et 20 *epoch*.

Méthodologie CNN

La création de l'architecture CNN a demandé beaucoup d'essais-erreurs avant d'arriver au résultat actuel. Le CNN est divisé en 3 couches de convolution. Ces 3 couches servent à extraire progressivement les caractéristiques de l'image. Le CNN possède ensuite une étape de global average pooling suivie d'un aplatissement (flatten), où les informations des couches de convolution sont mise en un vecteur d'une dimension. Finalement, le modèle possède deux couches FC pour analyser ce vecteur.

Tout d'abord, les couches de convolution sont séparées par des couches de pooling. Ces couches ont pour but de résumer les groupes de pixels. À chaque étape de pooling, la résolution spatiale est divisée par deux, ce qui permet de se faire une idée de l'image de façon plus globale.

Ensuite, j'ai ajouté une étape de dropout léger entre les deux couches denses, afin de réduire le surentraînement du modèle. Comme mon jeu de données est plutôt petit (975 images), j'avais beaucoup de problèmes de surentraînement. Un dropout de 0,2 permet de moins se suradapter aux données d'entraînement et m'a permis d'atteindre de légèrement meilleurs résultats lors des tests. J'ai aussi initialement eu beaucoup de problèmes avec le taux d'apprentissage et le CNN. Lorsque celui-ci était à 0,05, le modèle semblait partir dans tous les sens durant la validation.



J'ai donc finalement décidé de baisser le taux d'apprentissage à 0,001 pour le CNN, ce qui donnait de meilleurs résultats.

Analyse des résultats transfert d'apprentissage

Le transfert d'apprentissage a tout de même réussi à atteindre une précision de 81 % sur la détection de l'art IA. Ce modèle semble prometteur et pourrait sûrement être amélioré en ajoutant d'autres transformations d'augmentation de données, comme des rotations aléatoires aux images. Il était très difficile d'atteindre de meilleurs résultats. Le modèle semble toujours plafonner aux alentours d'une précision de 80 %.

	precision	recall	f1-score	support
AiArtData	0.81	0.81	0.81	79
RealArt	0.77	0.77	0.77	66
accuracy			0.79	145
macro avg	0.79	0.79	0.79	145
weighted avg	0.79	0.79	0.79	145

Analyse des resultats de CNN

Il a été vraiment difficile de réussir à faire monter la précision du CNN lors des tests au-dessus de la barre des 60%. Je pense que les différences entre l'art humain et l'art IA sont difficiles à distinguer au sein du jeu de données, car les tendances ne sont pas claires, surtout avec un aussi petit jeu de données. Je suis plutôt satisfait que, grâce à la réduction du taux d'apprentissage et à l'ajout du dropout, il ait tout de même été possible d'atteindre les 60 % de précision.

```
              precision    recall  f1-score   support

   AiArtData       0.61      0.80      0.69        79
     RealArt       0.62      0.39      0.48        66

    accuracy                           0.61       145
   macro avg       0.62      0.60      0.59       145
weighted avg       0.62      0.61      0.60       145
```

Comparaison des approches

Après avoir mis en place les deux approches, il est clair que les résultats du transfert d'apprentissage (ResNet-50) sont bien meilleurs. Le modèle était plus simple à manipuler et il était plus facile d'améliorer les performances en analysant les métriques. En comparaison, le CNN a demandé plus de réflexion pour finalement aboutir à beaucoup plus d'essais-erreurs. Je pense qu'il serait cependant possible d'atteindre de bons résultats avec l'architecture CNN en y consacrant plus de temps. Par exemple, je pense que l'ajout de transformations aléatoires sur le jeu d'entraînement aurait grandement aidé l'approche CNN, qui souffrait beaucoup de surentraînement.

Conclusion

Ces résultats m'ont surtout fait comprendre que le sujet que j'ai choisi, la détection d'art IA, est particulièrement difficile à résoudre. En effet, les différences entre l'art IA et l'art humain sont subtiles et je pense que cette subtilité demanderait un plus grand jeu de données d'entraînement. Je pense de plus que mon architecture CNN était trop simple pour une tâche pareille, qui demande beaucoup d'analyse des subtilités. Ce travail m'a cependant permis de bien comprendre les limitations et les différences entre les deux approches. S'il fallait améliorer le système, je pense que la prochaine étape serait d'ajouter des transformations d'augmentation sur le jeu de données lors du chargement.